

# TCP Connection Management and Flow Control

Module 11

Duration: 20:27

Video: <https://www.youtube.com/watch?v=TDYgEmXtWHc>

Course Coordinator - Dr Maninder Singh, Professor and Dean, Academic Affairs, Thapar University, Patiala

Subject Expert - Sunil Kumar, Department of Electrical and Computer Engineering, Punjab Agriculture University, Ludhiana

Auto-generated YouTube transcript for personal study. Spoken wording belongs to the original lecture. Follow the video link above; this file is not an official CEC/SWAYAM publication.

Hello friends I welcome you all to the video lecture series of cyber security and information security in this lecture we will study about TCP connection management and flow control first of all please the contents which I shall cover after covering brief introduction about TCP connection establishment I shall take up data transfer in TCP and last I shall take up TCP connection termination so now let us start TCP is a connection oriented transport protocol establishes a virtual path between the source and destination all of these segments belonging to a message are then sent over this virtual path using a single virtual pathway for the entire message facilitates the acknowledgement process as well as the retransmission of damaged or lost frames

you may wonder how TCP which uses the services of Ip a connectionless protocol can be connection oriented the point is that a TCP connection is virtual not physical TCP operates at a higher level TCP uses the services of Ip to deliver individual segments to the receiver but it controls the connection itself if a segment is lost or corrupted it is retransmitted unlike TCP IP is unaware of this retransmission if a segment arrives out of order TCP holds it until the missing segments arrive IP is unaware of this reordering in TCP connection oriented transmission requires three phases connection establishment data transfer and connection termination connection establishment TCP transmits data in full duplex mode when two tcps in two

machines are connected then they are able to send segments to each other simultaneously this implies that each party must initialize communication and get approval from the other party before any data are transferred three-way hand checking the connection establishment in TCP is called three-way handshaking in our example an application program called the client wants to make a connection with another application program called the server using TCP as the transport layer protocol the process starts with the server the server program tells its TCP that it is ready to accept a connection 3 minutes, 1 second this request is called a passive open although the server TCP is ready to accept a connection from any machine in

the world it cannot make the connection itself the client program issues a request for an active open a client that wishes to connect to an open server tells its TCP to connect to a particular server TCP can now start the three-way handshaking process as shown in the figure to show the process we use timelines each segment has values for all its header fields and perhaps for some of its option fields too however we show only the few fields necessary to understand each phase we show the sequence number the acknowledgement number the control flags and window size if relevant the three steps in this pH are as follows the client sends the first segment a sync segment in which only the sync flag is set this segment is for synchronization of sequence numbers the client in our example

chooses a random number as the first sequence number and sends this number to the server this sequence number is called the initial sequence number is S note that this segment does not contain any acknowledgement number it does not define the window size either a window size definition makes sense only when a segment includes an acknowledgement the segment can also include some options that we discuss later in the chapter note that the S YN segment is a control segment that carries no data however it consumes one sequence number when the data transfer starts the Isn is incremented by one we can say that the S YN segment carries no real data but we can think of it as containing one imaginary bite a sync

segment cannot carry data but it consumes one sequence number the server sends the second segment a s YN plus a segment with two flag bits set s YN and a this segment has the Dual Purpose first it is a s YN segment for communication in the other direction the server uses this segment to initialize the sequence number for numbering the bytes sent from the server to the client the server also acknowledges the receipt of the SN segment from the client by setting the a flag and displaying the next sequence number it expects to receive from the client because it contains an acknowledgement it also needs to define the receive window size rent as we will see in the flow control section the client sends the third

segment this is just an a segment it acknowledges the receipt of the second segment with the AK flag and analment number field noted the sequence number in this segment is the same as the one in the S YN segment the AK segment does not consume any sequence numbers the client must also Define the server window size some implementations allow this third segment in the connection phase to carry the first chunk of data from the client in this case the third segment must have a new sequence number showing the bite number of the first bite in the data in general the third segment usually does not carry data and consumes no sequence numbers simultaneous open a rare situation may occur when both processes issue an active open in this case both tcps transmit a SN plus a

segment to each other and one single connection is established between them S1 flooding attack the connection establishment procedure in TCP is susceptible to a serious security problem called Sy and flooding attack this happens when one or more malicious attackers send a large number of s segments to a server pretending that each of them is coming from a

different client by faking The Source IP addresses in the datagrams the server assuming that the clients are issuing an active open allocates the necessary sources such as creating 8 minutes, 1 second transfer control block TCB tables and setting timers the TCP server then sends the SN plus AK segments to the fake clients which are lost when the server waits for the Third Leg of the hand

shaking process H sources are allocated without being used if during this short period of time the number of SN segments is large the server eventually Runs Out of resources and may be unable to accept connection request from valid clients this is y and flooding attack belongs to a group of security attacks known as a denial of service attack in which an attacker monopolizes a system with so many service request that the system overloads and denies service to valid requests some implementations of TCP have strategies to to alleviate the effect of syn attack some have imposed a limit of connection requests during a specified period of time others try to filter out datagrams coming from unwanted Source addresses one recent strategy is to postpone resource allocation until the

server can verify that the connection request is coming from a valid IP address by using what is called a cookie data transfer after connection is established BYOD directional data transfer can take place the client and server can send data and acknowledgements in both directions the acknowledgement is piggybacked with the data figure shows an example in this example after connection is established the client sends 2,000 bytes of data in two segments the server then sends 2,000 bytes in one segment the client sends one more segment the first three segments carry both data and acknowledgement but the last segment carries only an acknowledgement because there is no more data to be sent note the values of the sequence and acknowledgement numbers the data segment sent by the client have the psh flag set

so that the server TCP tries to deliver data to the server process as soon as they are received we discuss the use of this flag in more detail later the segment from the server on the other hand does not set the push flag most TCP implementations have the option to set or not set this flag pushing data we saw that the sending TCP uses a buffer to store the stream of data coming from the sending application program the sending TCP can select the segment size the receiving TCP also buffers the data when they arrive and delivers them to the application program when the application program is ready or when it is convenient for the receiving TCP this type of flexibility increases the efficiency of TCP however there are occasions in which the application

program has no need for this flexibility for example consider an application program that communicates interactively with another application program on the other end the application program on one site wants to send a key stroke to the application at the other site and receive an immediate response delayed transmission and delayed delivery of data may not be acceptable by the application program TCP can handle such a situation the application program at the center can request a push operation this means that the sending TCP must not

wait for the window to be filled it must create a segment and send it immediately the sending TCP must also set the push bit to let the receiving TCP know that the segment includes data that must be delivered to the receiving application

program as soon as possible and not to wait for more data to come although the push operation can be requested by the application program most current TCP implementations ignore such requests TCP can choose whether or not to use this feature urgent data TCP is a stream oriented protocol this means that the data is presented from the application program to TCP as a stream of bytes each bite of data has a position in the Stream however there are occasions in which an application program needs to send urgent bytes some bites set need to be treated in a special way by the application at the other end the solution is to send a segment with the urg bit set the sending application program tells the sending

TCP that the piece of data is urgent the sending TCP creates a segment and inserts the Urgent data at the beginning of the segment the rest of the segment can contain normal data from the buffer the Urgent pointer field in the header defines the end of urgent data when they receiving TC P receives a segment with the urg bit set it informs the receiving application of the situation how this is done depends on the operating system it is then to discretion of the receiving program to take an action it is important to mention that tcp's urgent data is neither a priority service nor an expedited data service rather TCP urgent mode is a service by which the application program at the center side marks some portion of the

bite stream as needing special treatment by the application program at the receiver site thus signaling the presence of urgent data and marking its position in the data stream are the only aspects that distinguish the delivery of urgent data from the delivery of all other TCP data for all all other purposes urgent data is treated identically to the rest of the TCP by stream the application program at the receiver site must read every bite of data exactly in the order it was submitted regardless of whether or not urgent mode is used the standard TCP as implemented does not ever deliver any data out of order connection termination any of the two parties involved in exchanging data either client or either server can close the

connection although it is usually initiated by the client most implementations today allow two options for connection termination three-way handshaking and four-way handshaking with the half close option three-way handshaking most implementations today allow three-way handshaking for connection termination as shown in the figure as per figure I'll explain the working in three steps first in a common situation the client TCP after receiving a close command from the client process sends the first segment a f segment in which the F flag is set note that a f segment can include the lastar chunk of data sent by the client or it can be just a control segment as shown in the figure if it is only a control segment it

consumes only one sequence number the F segment consumes one sequence number if it does not carry data second the server TCP after receiving the F segment informs its process of the situation and sends the second segment a f plus a segment to confirm the receipt of the F segment from the client and at the same time to announce the the closing of the connection in the other direction this segment can also contain the last chunk of data from the server if it does not carry data it consumes only one sequence number third the client TP sends the last segment and a segment to confirm the receipt of the FY segment from the TCP server this segment contains the acknowledgement number which is 1 plus

the sequence number received in the FY segment from the server this segment cannot carry data and consumes no sequence numbers half close in TCP one and can stop sending data while still receiving data this is called a half close either the server or the client can issue a half close request it can occur when the server needs all the data before processing can begin a good example is sorting when the client sends data to the server to be sorted the server sends to receive all the data before sorting can start this means the client after sending all data can close the connection in the client to server Direction however the server to client direction must remain open to return the sorted data the server after receiving

the data still needs time for sorting its outbound direction must remain open figure shows an example of an half close in the figure data transfer from the client to the server stops the client half closes The Connection by sending a f segment the server accepts the half close by sending the AK segment the server however can still send data when the server has sent all of the processed data it sends a f i segment which is acknowledged by an a from the client after half closing the connection data can travel from the server to the client and acknowledgments can travel from the client to the server the client cannot send any more data to the server note the sequence numbers we have used the second segment consumes no

sequence number although the client has received sequence number y less than equal to 1 the server sequence number is still y greater than equal to 1 when the connection finally closes the sequence number of the last AK segment is still X because no sequence numbers are consumed during data transfer in that direction so friends this was all about TCP connection management and flow control hope the concepts explained in this lecture were understanding able and helpful hope to see you in the next lecture till then goodbye thank you